

Getting Started with Python Programming for DSP Labs

Jianhua Liu

Fall 2026

Contents

1	Introduction	1
2	Planning of the project	2
2.1	Project folders	2
2.2	CLI commands for different subprojects	2
2.3	GitHub repo	2
3	Creating the dsp26 project	2
3.1	Creating the project locally	2
3.2	Adding dependencies	3
3.3	Adding initial code directly under src/dsp26	3
3.3.1	The <code>__init__.py</code> file	3
3.3.2	The <code>app_cli.py</code> file	3
3.4	Adding lab code under src/dsp26/lab0_sinusoids	4
3.4.1	The <code>recipes.py</code> file	4
3.4.2	The <code>sinusoids.py</code> file	5
4	Running the project	7
5	Submission of work	7

1 Introduction

There will be many DSP Lab and class projects. Class projects include the following:

- Demo code of lectures.
- Code for HW problems.
- Class projects for graduate students.

As we discussed before, we will use Python for all the class related projects. These projects are mostly one or many Python script files.

One issue with Python projects is the dependency—the many existing libraries that used by source files of the projects. A common way to address the dependency issue is to create a virtual environment for each project.

There are many different ways to create virtual environments. Here, we use a modern tool, called uv, developed using Rust. See `py-pkg-1c-uv-cs` for details about how to install and use uv.

uv will put all the dependency libraries inside a `.venv` folder, which can contain library files up to a few Giga Bytes in size when there are many dependencies.

To strike a balance between the separation of projects and the size of storage used in a computer, we plan to have all the lab and class projects hosted in a single virtual environment. With this, the functionalities of all different projects of the DSP class are tied up by a single CLI app which can run in a terminal.

`py-pkg-1c-uv-cs` shows how to manage Python packages using uv; please read it carefully. We will use the same approach to create and manage a Python package for all lab and class projects for the DSP class.

2 Planning of the project

2.1 Project folders

- Use a single folder to host all source files. Yet, the folder structure is a bit different than a general-purpose project. For a general-purpose project, the source code files are organized according to their functionality. For convenience of DSP project submission, we organize the project files differently—we put the Python script files of each specific project into a single folder, which may contain subfolders.
- The root folder of the project is named `dsp26`.
- The parent folder of the `dsp26` folder is `projects`.

2.2 CLI commands for different subprojects

We will use the `app_cli.py` file to host the top level code for Typer. Each project folder under `src/dsp26` will corresponds to a command after `dsp26`. We can use Typer options to add options for each command. This way, we do not need to activate the venv to run the app in a terminal.

2.3 GitHub repo

This part should be skipped by students; it is only for the instructor to keep personal notes.

The local repo is in `dsp26`, and the GitHub repo is under `eraus` directly.

3 Creating the dsp26 project

3.1 Creating the project locally

We normally use Python that is two versions behind the newest release to make sure the dependencies work well. For projects created in 2026, where the newest release version is 3.14, we use Python 3.12.

On windows, this is done by running the following under the `projects` directory in a terminal: `uv init --package --python 3.12dsp26`.

3.2 Adding dependencies

We add two different kind of dependencies for this project.

The first is like Typer, used when both running the App or test various functions. Here, we run `uv add typer matplotlib numpy scipy`.

The second is like Pytest, used only during developing of the project. Here, we use `uv add --dev pytest` to install it only for the develop group.

3.3 Adding initial code directly under `src/dsp26`

3.3.1 The `__init__.py` file

This is the entry point of the app.

`__init__.py`:

```
1 from dsp26.app_cli import app
2
3 def main():
4     app()
```

3.3.2 The `app_cli.py` file

This is the collection of commands for this app although we only have one for this lab.

`app_cli.py`:

```
1 import typer
2 from dsp26.lab0_sinusoids.recipes import plot_sinusoids_td_fd_
3
4
5 app = typer.Typer()
6
7 __version__ = "0.1.0"
8
9
10 #####
11 # Print out the version: numreps --version
12 def print_version(value: bool):
13     if value:
14         typer.echo(f"Version of dsp26: {__version__}")
15         raise typer.Exit()
16
17
18 # Add the version option.
19 @app.callback()
20 def callback(version: bool = typer.Option(
21     None, "--version", callback=print_version, is_eager=True)):
22     pass
23
24
25 #####
26 # Lab0-sinusoids
27 @app.command()
```

```

28 def plot_sinusoids_td_fd(
29     folder_name: str = typer.Option(
30         "",
31         "--folder-name",
32         help="Optional output folder name for saving generated plots. " \
33             "If set, plots will be written under <folder-name>/figs.",
34     ),
35 ):
36     """Plot time-domain waveform and frequency-domain spectrum of sinusoids.
37     Use --folder-name to save figures to <folder-name>/figs."""
38     plot_sinusoids_td_fd(folder_name=folder_name)

```

3.4 Adding lab code under `src/dsp26/lab0_sinusoids`

We have two files here.

3.4.1 The `recipes.py` file

This is the file used to bridge the commands in the `app_cli.py` file.

`recipes.py`:

```

1  import numpy as np
2
3  from dsp26.lab0_sinusoids.sinusoids import (
4      plot_one_sinusoid_td_fd,
5      plot_multi_sinusoids_td_fd_sum,
6      plot_multi_sinusoids_td_fd_parameters,
7  )
8
9
10 myID = 1234 # last 4 digits of my ERAU ID
11 np.random.seed(seed=myID) # set the seed of the random number generator
12
13 pi = np.pi
14 # (random) frequency of signal in Hz
15 Freq_1 = np.floor((10 + np.random.randint(0,10)) / 3)
16 print("The basic frequency is", Freq_1, "Hz.")
17
18 Nd = 1024 # total number of samples for dense sampling; d = dense
19 Ns = Nd / 16 # total number of samples for sparse sampling; s = sparse
20 Ns2 = int(Ns // 2) # Ns/2, to be used as the index for array
21 Fs_d = Nd # sample rate for dense sampling
22 Fs_s = Ns # sample rate for sparse sampling
23 t_d = np.arange(Nd) / Fs_d # sample times for dense sampling
24 t_s = np.arange(Ns) / Fs_s # sample times for sparse sampling
25 Freq_s = np.arange(-Fs_s/2, Fs_s/2) # frequencies for spectrum display
26
27 Am = np.array([1, 0.5, 0.2]) # amplitudes of three signals
28 Fm = np.array([Freq_1, 2*Freq_1, 3*Freq_1]) # frequencies of three signals
29 Pm = np.array([np.pi/4, np.pi/3, np.pi/9]) # phases of three signals
30
31
32 def plot_sinusoids_td_fd(folder_name: str = ""):
33     A1, F1, P1 = 1, Freq_1, np.pi/4
34     fig1, x1_d, x1_s = plot_one_sinusoid_td_fd(A1, F1, P1, t_d, t_s, Ns, Fs_s,
35         Freq_s)

```

```

35
36     A2, F2, P2 = 0.5, 2*Freq_1, np.pi/3
37     fig2, x2_d, x2_s = plot_one_sinusoid_td_fd(A2, F2, P2, t_d, t_s, Ns, Fs_s,
38         Freq_s)
39
40     x_s = x1_s + x2_s
41     x_d = x1_d + x2_d
42     fig3 = plot_multi_sinusoids_td_fd_sum(x_d, x_s, t_d, t_s, Ns2, Freq_s)
43
44     fig4 = plot_multi_sinusoids_td_fd_parameters(Am, Fm, Pm, t_d, t_s, Ns2,
45         Freq_s)
46
47     if folder_name:
48         from pathlib import Path
49
50         figs_dir = Path(folder_name) / "figs"
51         figs_dir.mkdir(parents=True, exist_ok=True)
52
53         fig1.savefig(figs_dir / "lab0_sinusoids_fig1.svg")
54         fig2.savefig(figs_dir / "lab0_sinusoids_fig2.svg")
55         fig3.savefig(figs_dir / "lab0_sinusoids_fig3.svg")
56         fig4.savefig(figs_dir / "lab0_sinusoids_fig4.svg")
57         print(f"Figures Saved in {folder_name}")

```

3.4.2 The `sinusoids.py` file

This is the collection of all the lower level functions used for plotting and time-domain waveform and frequency-domain spectrum.

`sinusoids.py`:

```

1  import numpy as np
2  import matplotlib.pyplot as plt
3  from scipy import signal
4
5
6  # Plot the waveform and spectrum of a single frequency sinusoidal
7  # signal with Am, Fr, and Ph.
8  def plot_one_sinusoid_td_fd(Am, Fr, Ph, t_d, t_s, Ns, Fs_s, Freq_s):
9      x_s = Am * np.cos(2*np.pi*Fr*t_s + Ph)
10     x_d = Am * np.cos(2*np.pi*Fr*t_d + Ph)
11     X_s = np.fft.fft(x_s)
12     X_F = np.abs(np.fft.fftshift(X_s))
13     Freqint1 = int(np.round(Fr*Ns/Fs_s))
14
15     fig_plot = plt.figure(1)
16     plt.subplot(211)
17     plt.plot(t_s, x_s, 'o', t_d, x_d)
18     plt.axis([0, t_d[-1], -1.5, +1.5])
19     plt.xlabel('Time')
20     plt.ylabel('Waveform')
21
22     fig_text = f"{X_s[Freqint1].real:.3e} + j{X_s[Freqint1].imag:.3e}"
23     plt.subplot(212)
24     plt.stem(Freq_s, X_F)
25     plt.text(Fr-2, np.abs(X_s[Freqint1])+3, fig_text)
26     plt.axis([Freq_s[0], Freq_s[-1], 0, 45])

```

```

27     plt.xlabel('Freq')
28     plt.ylabel('Spectrum')
29
30     fig = plt.gcf()
31     fig.set_size_inches(10, 10)
32     plt.show()
33     return fig_plot, x_d, x_s
34
35
36 # Plot the waveform and spectrum of the superposition of multiple
37 # sinusoidal signals given the summed signals
38 def plot_multi_sinusoids_td_fd_sum(x_d, x_s, t_d, t_s, Ns2, Freq_s):
39     X_s = np.fft.fft(x_s)
40     X_F = np.abs(np.fft.fftshift(X_s))
41     peakinds, _ = signal.find_peaks(np.abs(X_s[0:Ns2]), height=1)
42
43     fig_plot = plt.figure(3)
44     plt.subplot(211)
45     plt.plot(t_s, x_s, 'o', t_d, x_d)
46     plt.axis([0, t_d[-1], -1.5, +1.5])
47     plt.xlabel('Time')
48     plt.ylabel('Waveform')
49
50     plt.subplot(212)
51     plt.stem(Freq_s, X_F)
52     for peak in peakinds:
53         fig_text = f"{X_s[peak].real:.3e} + j{X_s[peak].imag:.3e}"
54         plt.text(peak-2, np.abs(X_s[peak])+3, fig_text)
55     plt.axis([Freq_s[0], Freq_s[-1], 0, 45])
56     plt.xlabel('Freq')
57     plt.ylabel('Spectrum')
58
59     fig = plt.gcf()
60     fig.set_size_inches(10, 10)
61     plt.show()
62     return fig_plot
63
64
65 # Plot the waveform and spectrum of the superposition of multiple
66 # sinusoidal signals given the parameter vectors
67 def plot_multi_sinusoids_td_fd_parameters(Am, Fm, Pm, t_d, t_s, Ns2, Freq_s):
68     # Generate the sparse sequence using the conventional method
69     x_s = np.zeros_like(t_s)
70     for i in range(len(Am)):
71         x_s += Am[i] * np.cos(2*np.pi*Fm[i]*t_s + Pm[i])
72
73     # Generate the dense sequence using the SUM methon
74     x_d = np.sum(
75         Am.reshape(1,-1)
76         * np.cos(2*np.pi*np.dot(t_d.reshape(-1,1),Fm.reshape(1,-1))
77             + Pm.reshape(1,-1)),
78         axis=1
79     )
80
81     X_s = np.fft.fft(x_s)
82     X_F = np.abs(np.fft.fftshift(X_s))
83     peakinds, _ = signal.find_peaks(np.abs(X_s[0:Ns2]), height=1)
84

```

```
85     fig_plot = plt.figure(4)
86     plt.subplot(211)
87     plt.plot(t_s, x_s, 'o', t_d, x_d)
88     plt.axis([0, t_d[-1], -1.5, +1.5])
89     plt.xlabel('Time')
90     plt.ylabel('Waveform')
91
92     plt.subplot(212)
93     plt.stem(Freq_s, X_F)
94     for peak in peakinds:
95         fig_text = f"{X_s[peak].real:.3e} + j{X_s[peak].imag:.3e}"
96         plt.text(peak-2, np.abs(X_s[peak])+3, fig_text)
97     plt.axis([Freq_s[0], Freq_s[-1], 0, 45])
98     plt.xlabel('Freq')
99     plt.ylabel('Spectrum')
100
101     fig = plt.gcf()
102     fig.set_size_inches(10, 10)
103     plt.show()
104     return fig_plot
```

4 Running the project

Before running the project, you have to install it; check the instruction in `py-pkg-1c-uv-cs`.

To see what commands are provided by the project, you can run `dsp26--help`. With that, you can run the command to plot four different figures.

5 Submission of work

This is Lab 0, and the submission is only for a testing if you can run the project.

The focus of this submission is the screenshot of the four figures. Note that these figures can be saved to files using an option of the command of the project.